1. Create a class called Playlist with a default constructor that sets the playlist name to 'My Favourites' and prints a welcome message when an object is created.

```
class Playlist:
  def __init__(self):
    self.name = "My Favourites"
    print("Welcome to your Playlist!")

p1 = Playlist()
print("Playlist Name:", p1.name)
```

Output:

Welcome to your Playlist!

Playlist Name: My Favourites

2. Build a class Product for a Flipkart-style app with a parameterized constructor that takes productName, price, and rating as arguments and displays all details using a displayInfo() method.

```
class Product:
  def __init__(self, productName, price, rating):
    self.productName = productName
    self.price = price
    self.rating = rating

  def displayInfo(self):
    print("Product Name:", self.productName)
    print("Price:", self.price)
    print("Rating:", self.rating)

p1 = Product("Mobile Phone", 15000, 4.5)
```

p1.displayInfo()

Output:

Product Name: Mobile Phone

Price: 15000

Rating: 4.5

3. Create a class Movie with a parameterized constructor and then use a copy constructor to duplicate a Movie object. Print both the original and copied movie details to show they are identical.

Python does not have a built-in copy constructor like C++, so we can create one using an optional object parameter.

```
class Movie:
  def __init__(self, name, rating=None):
    if isinstance(name, Movie):
      self.name = name.name
      self.rating = name.rating
    else:
      self.name = name
      self.rating = rating

  def displayInfo(self):
    print("Movie Name:", self.name)
    print("Rating:", self.rating)

m1 = Movie("Avatar", 8.5)

m2 = Movie(m1)

print("Original Movie:")
```

```
m1.displayInfo()


print("Copied Movie:")
m2.displayInfo()
```

Output:

Original Movie:

Movie Name: Avatar

Rating: 8.5

Copied Movie:

Movie Name: Avatar

Rating: 8.5

**4. Simulate a BookMyShow ticket booking by creating a Ticket class that prints 'Saving your ticket...' in its destructor. Create and delete a Ticket object to demonstrate the destructor lifecycle.

Hint: Use the __del__ method (Python) or ~Ticket() (C++) to define the destructor.**

```
class Ticket:
  def __init__(self):
    print("Ticket booked successfully!")

  def __del__(self):
    print("Saving your ticket...")

t1 = Ticket()

del t1
```

Output:

Ticket booked successfully!

Saving your ticket...

Explanation: __del__() is a destructor method. It may run when an object is destroyed or garbage-collected. In this example, del t1 removes the reference to the object, and the destructor runs when the object is actually finalized.

**5. Modify your Playlist class so that it auto-saves the playlist name to a file called autosave.txt when the object is destroyed, simulating an auto-save feature like Spotify.

Hint: Write the file-saving code inside the destructor.**

```
class Playlist:
  def __init__(self, name):
    self.name = name
    print("Playlist created:", self.name)

  def __del__(self):
    with open("autosave.txt", "w") as file:
      file.write(self.name)
    print("Playlist saved successfully!")

p1 = Playlist("My Favourites")

del p1
```

Output (normally):

Playlist created: My Favourites

Playlist saved successfully!

The program creates a file named autosave.txt containing:

My Favourites